

Machine Learning

Leonardo Ciscato Pajello
Trentin Tommaso

July 23, 2025

Contents

1	Introduzione	2
1.1	Reazione con l'AI e il Deep learning	2
1.2	Applicazioni	2
2	Tipi di Apprendimento Automatico	2
2.1	Apprendimento Supervisionato	2
2.1.1	Regressione	2
2.1.2	Regressione Lineare	2
2.1.3	Regressione Logistica	3
2.1.4	Normalizzazione	3
2.1.5	Pearson	3
2.1.6	Classificazione	4
2.1.7	Support Vector Machine	4
2.2	Apprendimento Non Supervisionato	4
2.2.1	Clustering	4
2.2.2	K-Means	5
2.2.3	Clustering Gerarchico Agglomerativo	5
2.2.4	DBSCAN	5
2.3	Apprendimento Rinforzato	5
2.4	Apprendimento Auto-Supervisionato	5
2.5	Content Generations	5
3	Diagnostica e Miglioramento dei Modelli	6
3.1	Over e Under fitting	6
3.1.1	Overfitting (High-variance)	6
3.1.2	Underfitting (High-bias)	6
3.2	Regularization	6
3.3	Curve di Apprendimento	6
3.4	Azioni per migliorare le Prestazioni	6
3.5	Metriche di Performance	6
4	Reti Neurali	7
4.1	Funzioni di Attivazione	7
4.2	Addestramento	7
4.3	Funzioni di Loss	8
5	Lavorare con Dati Testuali	8
5.1	Classificazione del Testo	8
5.2	Preprocessing	8
5.3	Rappresentazioni del Testo	8
5.3.1	Bag of Words	8
5.3.2	TF-IDF	8
5.3.3	Word Representations	8
6	Riduzione della Dimensionalità	9
6.1	PCA	9
6.2	Scelta di k	9
7	Rilevamento delle Anomalie	10

1 Introduzione

Definizione: Il Machine Learning è un **campo di studio che conferisce ai computer la capacità di imparare senza essere esplicitamente programmati**. Si dice che un programma impari da un'esperienza E, rispetto ad una task T e misurazioni di performance P, se la sua performance su T, misurata con P migliora con esperienza E. un esempio è un programma di posta elettronica che impara a filtrare lo SPAM osservando come etichetti le e-mail.

1.1 Reazione con l'AI e il Deep learning

- L'**AI** include qualsiasi tecnica che consenta ai computer di **mimare** il comportamento umano;
- Il **ML** è un **sottoinsieme delle tecniche** di AI che utilizzano metodi statistici per consentire alle macchine di migliorare con l'esperienza;
- Il **Deep Learning** è un sottoinsieme del **ML** che rende fattibile il calcolo di reti neurali multistrato;

1.2 Applicazioni

Il ML è parte della nostra vita quotidiana, con applicazioni come il riconoscimento della scrittura a mano, il Natural Language Processing (NLP), la Computer Vision, l'analisi di database di grandi dimensioni (dati di clic web, cartelle cliniche) e programmi che si autopersonalizzano come raccomandazioni Amazon o Netflix.

2 Tipi di Apprendimento Automatico

Gli algoritmi di ML possono essere suddivisi principalmente in:

2.1 Apprendimento Supervisionato

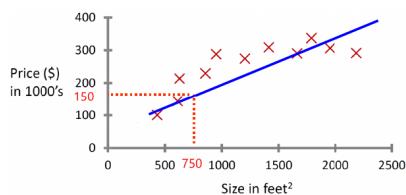
Supervised Learning che si basa su dataset in cui la risposta giusta è già *fornita*, ovvero i dati sono etichettati. L'obiettivo è imparare una funzione (ipotesi) che mappa gli input agli output desiderati.

2.1.1 Regressione

Predizione di un **valore continuo** come output, cercando il migliore modello che applicabile ai dati forniti (ex. il prezzo di una casa basato sulla sua dimensione).

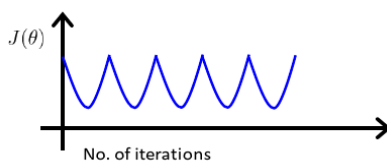
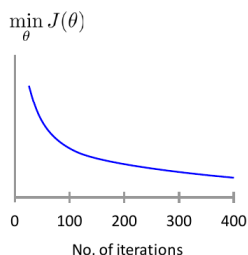
2.1.2 Regressione Lineare

Modella la relazione tra input e output con una *linea retta* o con un *iperpiano* nel caso delle polinomiali (in questo caso si utilizzano le medesime formule con le variabili elevate alle rispettive potenze). L'obiettivo è **minimizzare** una funzione di costo (generalmente l'errore quadratico medio) che misura quanto bene l'ipotesi si adatti ai dati. L'algoritmo di **Gradient Descent** è utilizzato per trovare i parametri ottimali che minimizzano questa funzione.



Formule

- **Ipotesi:** $h_{\theta}(x) = \theta_0 + \theta_1 x_1 + \dots + \theta_n x_n = \theta^T x$
- **Funzione di costo:** $J(\theta) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x_i) - y_i)^2$
- **Gradient Descent - Formula Generica:** $\theta'_j = \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta)$
- **Gradient Descent - θ_0 :** $\theta'_0 = \theta_0 - \alpha \frac{1}{n} \sum_{i=1}^m (h_{\theta}(x_i) - y_i)$
- **Gradient Descent - θ_1 :** $\theta'_1 = \theta_1 - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x_i) - y_i) x_i$

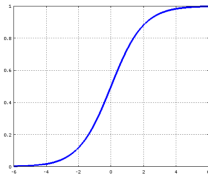


Gradient descent not working.

Figure 1: Gradient Descent

2.1.3 Regressione Logistica

E' sempre un algoritmo di classificazione, utilizza la funzione *Sigmoide* per mappare le predizioni nell'intervallo (0,1), rappresentando la probabilità di appartenenza ad una classe. Definisce un **decision boundary** che separa le classi, e può essere estesa per la **classificazione multi-classe** utilizzando l'approccio *One VS All*;



Formule

- **Sigmoid:** $\frac{1}{1+e^{-z}}$
- **Ipotesi:** $h_{\theta} = g(\theta^T x) = \frac{1}{1+e^{-\theta^T x}}$
- **Cost:** $-y \log(h_{\theta}(x)) - (1 - y) \log(1 - h_{\theta}(x))$
- **Funzione di Costo:** $J(\theta) = \frac{1}{m} \sum_{i=1}^m \text{Cost}(h_{\theta}(x_i), y_i)$

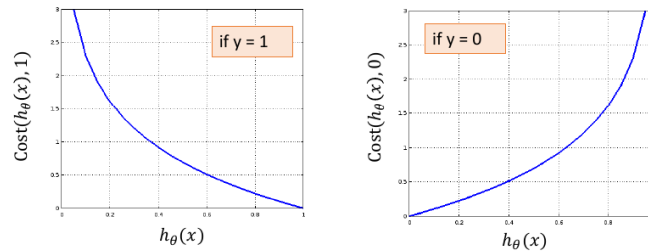


Figure 2: Cost Func Logistic Regression

2.1.4 Normalizzazione

Si normalizzano le features nella **Feature Normalization**, questo è fondamentale per garantire che le caratteristiche siano su una scala simile, migliorando così la convergenza degli algoritmi di ottimizzazione, i metodi trattati sono:

- **Mean Normalization**

Formule

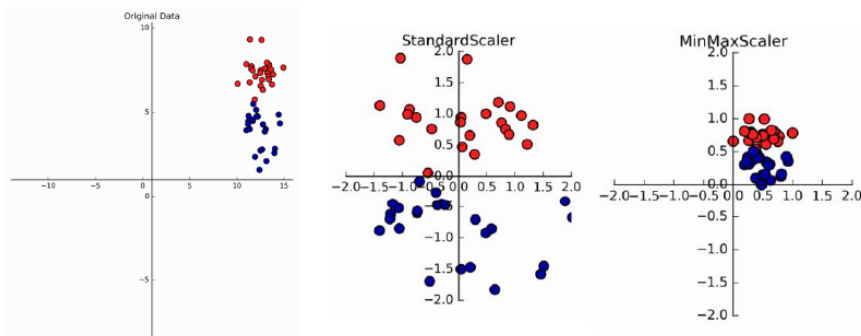
- **Media:** $\mu_x = \frac{1}{m} \sum_{i=1}^m x_i$
- **Deviazione standard:** $\sigma = \sqrt{\frac{1}{m-1} \sum_{i=1}^m (x_i - \mu_x)^2}$
- **Valore normalizzato:** $x'_i = \frac{x_i - \mu_x}{\sigma}$

- **Min Max Normalization**

Formula

- **Valore normalizzato:** $x'_i = \frac{x_i - \min(x)}{\max(x) - \min(x)}$

E' di cruciale importanza applicare la stessa standardizzazione sia al training set che al test set. I calcoli di media e varianza vanno fatti **solo** sul training set.



2.1.5 Pearson

Il coefficiente di correlazione di Pearson (indicato con ρ o r) misura la relazione lineare tra due variabili. Assume valori compresi tra -1 e 1:

- +1 indica una correlazione lineare positiva perfetta,
- 0 indica assenza di correlazione lineare,
- -1 indica una correlazione lineare negativa perfetta.

Questo coefficiente è utile per analizzare la dipendenza tra due caratteristiche e può aiutare a identificare variabili ridondanti o fortemente correlate, migliorando così la selezione delle feature.

Formula

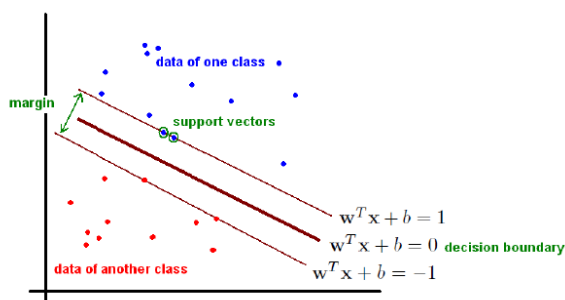
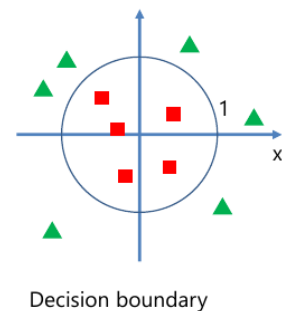
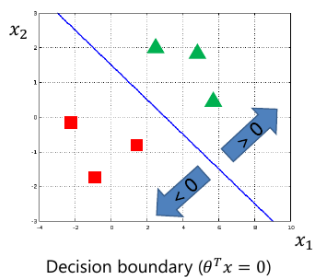
- **Media:** $\mu_{x/y} = \frac{1}{N} \sum_{i=1}^N x_i/y_i$
- **Deviazione Standard:** $\sigma_{x/y} = \sqrt{\frac{1}{N-1} \sum_{i=1}^N (x_i/y_i - \mu_{x/y})^2}$
- **Coefficiente di Pearson** $\rho = \frac{1}{N-1} \sum_{i=1}^N \left(\frac{x_i - \mu_x}{\sigma_x} \right) \left(\frac{y_i - \mu_y}{\sigma_y} \right)$

2.1.6 Classificazione

La classificazione predice un **valore discreto**: i suoi sistemi cercano, dato un insieme di punti di trovare il *decision boundary* appropriato per dividerli (ex. Spam - Not Spam / Tumore Maligno - Benigno).

2.1.7 Support Vector Machine

L'SVM mira a trovare il **decision boundary** che massimizzi il margine tra le classi, cioè la distanza dal confine ai punti dati più vicini di ciascuna classe. Anche in presenza di dati non perfettamente separabili si introduce il concetto di **margine soft**, tramite le variabili di slack e il parametro C che controlla la penalità per le *missclassifications*. Le **Kernel functions** consentono agli SVM di trovare **decision boundary** non lineari.



Formule: sia $\exp$: e

- **Decision Boundary:** $W^T x + b = 0$
- **Margine Superiore:** $W^T x + b = 1$
- **Margine Inferiore:** $W^T x + b = -1$
- **Distanza tra margini:** $\lambda \|W\| = \frac{2}{\sqrt{W^T W}}$
- **Funzione di costo:** $\min_{W,b} \frac{W^T W}{2}$
(soggetta a $y_i(W^T x_i + b) \geq 1 \forall x_i$)
- **Kernel Gaussiano:** $f1 = \text{similarity}(x, l^1) = \exp\left[-\frac{\sum_{j=1}^n (x_j - l_j^1)^2}{2\sigma^2}\right]$

2.2 Apprendimento Non Supervisionato

L'**Unsupervised Learning** ha l'obiettivo di trovare una struttura o dei raggruppamenti nei dati, in questo caso i dati non sono etichettati.

2.2.1 Clustering

Il Clustering prevede algoritmi di raggruppamento di dati in **cluster coesivi**

2.2.2 K-Means

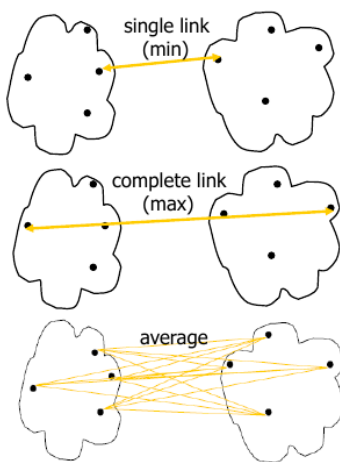
Questo algoritmo iterativo raggruppa i dati in K cluster predefiniti. Inizializza casualmente i centriodi dei cluster e poi alterna l'assegnazione dei punti ai centriodi più vicini e l'aggiornamento dei centriodi come media dei punti assegnati. Può convergere a un **ottimo locale** per cui si consiglia l'inizializzazione causale multipla.

Formule:

- **Distanza Euclidea:** $d = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$
- **Nuovo Centriode** di un cluster Cl_i : $Cl_i = (\frac{1}{n} \sum_{j=1}^n x_i \in Cl_i; \frac{1}{n} \sum_{j=1}^n y_i \in Cl_i)$

2.2.3 Clustering Gerarchico Agglomerativo

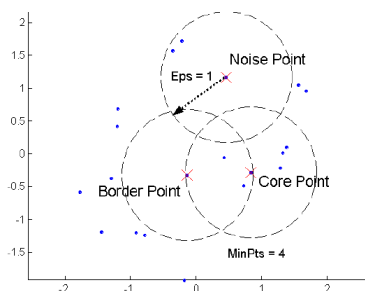
Herarchical Agglomerative Clustering è un metodo "bottom-up" che inizia con ogni punto dati in un cluster separato e **fonde iterativamente** le coppie di cluster più **simili**, fino ad ottenere un unico grande cluster. Questo produce un *dendrogramma* che mostra la gerarchia dei raggruppamenti. Utilizza diverse **misure di distanza** tra cluster come:



- **single link:** calcola la distanza minore tra i cluster;
- **complete link:** calcola la distanza maggiore tra cluster più vicini;
- **average link:** calcola la distanza media tra cluster più vicini;

2.2.4 DBSCAN

L'acronimo sta per "Density-Based Spatial Clustering of Applications with Noise", è un algoritmo di clustering basato sulla densità che può **scoprire cluster** di forma arbitraria e gestire il rumore. Classifica i punti come



- **Core point:** se hanno sufficienti vicini entro un raggio definito;
- **Border point:** se sono vicini ad un core point ma non core di per sé;
- **Noise point:** punti esterni;

2.3 Apprendimento Rinforzato

Reinforced Learning, in questo caso un agente impara a prendere decisioni in un ambiente per **massimizzare una ricompensa**. Esempi includono la robotica e il gioco del Go (Alpha Go di DeepMind).

2.4 Apprendimento Auto-Supervisionato

Self Supervised Learning non molto approfondito nei pdf.

2.5 Content Generations

Questa è un'area emergente che include le **Generative Adversarial Networks** impiegate per creare nuovi contenuti. Questi si possono immaginare come vettori di pixel plottati in spazi ad altissima dimensionalità, lo si fa per tutte

le immagini date come esempio, che possono raffigurare anche oggetti diversi, poi se si spostano su un piano 2D, si possono notare i gruppi distinti di punti associati alle immagini relative. Per creare un nuovo oggetto si può condizionare l'algoritmo per identificare la zona potenzialmente corretta nel grafico, in quanto non tutti i punti sono eligibili per generare l'oggetto desiderato.

3 Diagnostica e Miglioramento dei Modelli

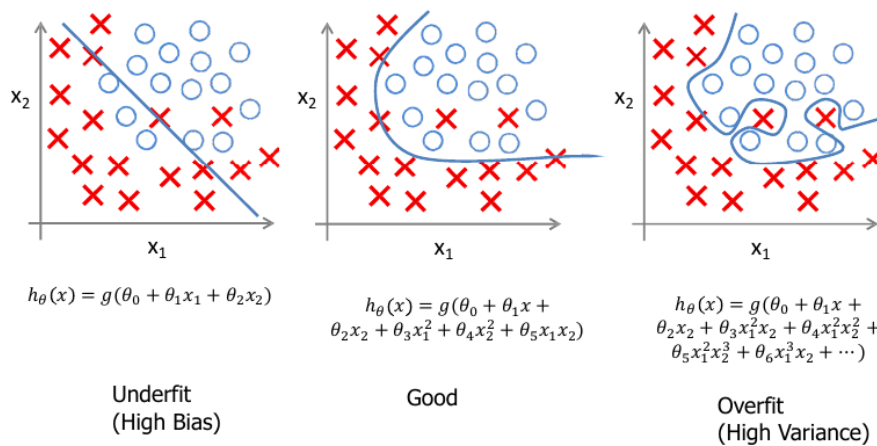
3.1 Over e Under fitting

3.1.1 Overfitting (High-variance)

Il modello si adatta **troppo bene** al training set, ma non riesce a generalizzare a nuovi esempi. Tipicamente l'errore sul training set è basso ma la validazione porta ad un errore alto e continua a diminuire con più dati.

3.1.2 Underfitting (High-bias)

Il modello troppo semplice **non riesce** a catturare le relazioni sottostanti nei dati, nè sul training set nè sui nuovi esempi. Tipicamente sia l'errore di training che quello di validazione sono alti e vicini tra loro.



3.2 Regularization

E' una tecnica per **ridurre l'overfitting** mantenendo tutte le caratteristiche ma riducendo la **magnitudine** dei parametri. Viene aggiunto un termine di penalità alla funzione di costo. Ex. Ridge Regression per la regressione lineare, o un termine simile per la logistica.

Il parametro λ (o C in scikit-learn) controlla la forza della regolarizzazione:

- Un λ grande riduce molto i parametri, riducendo la varianza ma aumentando il bias;
- Un λ piccolo permette ai parametri di avere valori maggiori;

Formule di costo regolarizzate per Linear e Logistic:

- **Regressione Lineare:** $J(\theta) = \frac{1}{2m} [\sum_1^m (h_{\theta}(x_i) - y_i)^2 + \lambda \sum_{j=1}^n \theta_j^2]$
- **Regressione Logistica:** $J(\theta) = \frac{1}{m} [\sum_1^m Cost(h_{\theta}(x), y) + \frac{\lambda}{2} \sum_{j=1}^n \theta_j^2]$

3.3 Curve di Apprendimento

Le **Learning Curves** sono grafici dell'errore di training e di validazione in funzione della dimensione del training set. Utili a **diagnosticare** problemi di bias o varianza.

3.4 Azioni per migliorare le Prestazioni

A seconda che il problema sia di alto Bias o alta Varianza si possono intraprendere diverse azioni, come ottenere più esempi di training, ridurre / aumentare le caratteristiche, o modificare λ .

3.5 Metriche di Performance

Per poter valutare i modelli di classificazione si usa la **Matrice di confusione** e da essa si derivano:

- **True Positive:** Predetto positivo $\rightarrow$ è vero;
- **True Negative:** Predetto negativo $\rightarrow$ è vero;
- **False Positive $\Rightarrow$ Errore di tipo 1:** Predetto positivo $\rightarrow$ è falso;
- **False Negative $\Rightarrow$ Errore di tipo 2:** Predetto negativo $\rightarrow$ è falso;
- **Accuracy:** $\frac{TP+TN}{TP+TN+FP+FN}$ è descrittiva in problemi sbilanciati;
- **Precision:** $\frac{TP}{TP+FP}$ frazione di istanze rilevanti tra quelle recuperate;
- **Recall:** $\frac{TP}{TP+FN}$ frazione di istanze rilevanti che non sono state recuperate;
- **F-Score:** $\frac{P \cdot R}{P+R}$ media aritmetica di Precision e Recall;

		Predicted class	
		P	N
Actual Class	P	True Positives (TP)	False Negatives (FN)
	N	False Positives (FP)	True Negatives (TN)

4 Reti Neurali

L'architettura delle **Neural Network** sono ispirate ad un cervello, composte da **strati** come: input, hidden, o output di **neuroni** o unità. Ogni connessione tra neuroni ha un **peso** (w) e ogni neurone ha un **bias** (b).

Layer: Siano W: pesi, b: bias e g: sigmoide(z)

1. $z_1 = W_1x + b_1$; $a_1 = g(z_1)$
2. $z_2 = W_2a_1 + b_2$; $a_2 = g(z_2)$
3. la formula dell'ultimo livello dipende dalla **funzione di attivazione**;

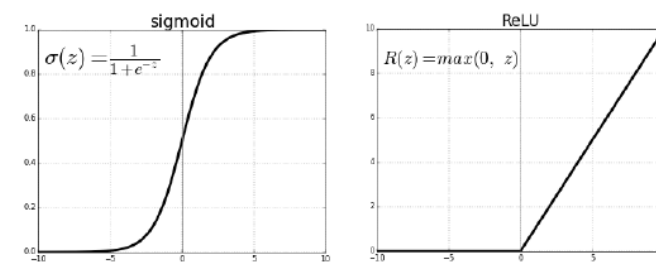
4.1 Funzioni di Attivazione

Le **Activation Functions** introducono non linearità nel modello, permettendo di apprendere relazioni complesse. Queste includono:

- **Sigmoid:** storicamente usata, ma con problemi di "saturazione del gradiente";
- **Rectified Linear Unit:** o RELU usata più comunemente oggi per la sua semplicità e accelerazione della convergenza;
- **Softmax:** usata per la classificazione multi-classe, l'ultimo strato spesso la usa per produrre una distribuzione di probabilità sulle classi;

Formule: sia $z = -\theta^t x = -(\theta_0 + \theta_1 x_1 + \dots + \theta_n x_n)$ (sigmoide), e N il numero di classi (SoftMax)

- **Sigmoide:** $g(z) = \frac{1}{1+e^{-z}}$
- **RELU:** se $z \leq 0$ ritorna 0, se $z > 0$ ritorna z
- **SoftMax:** $a_i = \frac{e^{z_i}}{\sum_{j=1}^N e^{z_j}}$



4.2 Addestramento

1. I pesi vengono inizializzati casualmente;
2. L'algoritmo di **Backpropagation** è utilizzato per addestrare la rete, consiste in due passi ripetuti:
 - (a) Passo in Avanti: I dati vengono propagati attraverso la rete per calcolare l'output e l'errore;
 - (b) Passo Indietro: L'errore viene propagato all'indietro attraverso la rete per aggiornare i pesi e i bias;

4.3 Funzioni di Loss

Queste misurano l'errore del modello, si dividono per le varie categorie:

- Per la classificazione binaria: **Binary Cross-Entropy Loss**;
- Per la regressione: **Mean Squared Error Loss (MSE)**;
- Per la classificazione multi-classe: **Cross-Entropy Loss**, che qualifica la differenza tra la distribuzione di probabilità predetta e quella reale.

Formule: sia m il numero di esempi contenuti nel training set (BCE, MSE); n : numero di esempi nel dataset, y_i distribuzione vera (one hot) per l'esempio i e $h_{W,b}(x_i)$ la distribuzione predetta dalla rete per l'input x_i (Cross-Entropy loss).

- **BCross-Entropy:** $L(W, b) = \frac{1}{m} \sum_{i=1}^m [-y_i * \log(h_{W,b}(x_i)) - (1 - y_i) * \log(1 - h_{W,b}(x_i))]$
- **MSE:** $L(W, b) = \frac{1}{m} \sum_{i=1}^m (h_{W,b}(x_i) - y_i)^2$
- **CE Multi-class:** $L(W, b) = - \sum_{i=1}^m y_i * \log(h_{W,b}(x_i))$

5 Lavorare con Dati Testuali

5.1 Classificazione del Testo

Applicazioni comuni includono l'analisi del sentiment.

5.2 Preprocessing

Sono fasi cruciali per preparare il testo quali:

- **Lower-casing:** Conversione dell'intero testo in minuscolo;
- **Word Stemming:** Riduzione delle parole alla loro "radice" tramite regole euristiche;
- **Lemmatization:** Raggruppamento delle forme flesse di una parola nella sua forma base o "lemma", richiede un dizionario e considera il contesto. Questo è più computazionalmente costoso dello stemming;
- **Stop Words:** Rimozione delle parole più comuni che hanno poco valore semantico ("the, is");
- **Punteggiatura e Regole speciali:** Rimozione della punteggiatura o normalizzazione basata sul dominio usando espressioni regolari.

5.3 Rappresentazioni del Testo

5.3.1 Bag of Words

La BoW rappresenta un documento come un **vettore di frequenza** di parole presenti in un vocabolario predefinito. La limitazione principale è che non **cattura l'ordine o il contesto** delle parole, quindi frasi con significati diversi possono avere la stessa rappresentazione in BoW. Per poter calcolare il vettore si contano le ricorrenze della parola cercata.

5.3.2 TF-IDF

la Term Frequency - Inverse Document Frequency è una tecnica che assegna un peso a ogni parola, basato sulla sua frequenza nel documento (TF) e sulla sua rarità tra tutti i documenti (IDF). Parole più **uniche** e **frequenti** in un documento avranno peso **maggiore**, mentre le parole più comuni (come le stop words) avranno peso minore.

Formula: siano N : numero totale di documenti, N_w : numero di documenti in cui appare w e tf : frequenza del termine w nel documento d .

- $tfidf(w, d) = tf(\ln(\frac{N+1}{N_w+1}) + 1)$

5.3.3 Word Representations

Sono tecniche più avanzate che rappresentano le parole come **vettori numerici** densi che **catturano** il loro significato semantico. Parole con significati simili sono posizionate vicino nello spazio vettoriale. Questo approccio supera le limitazioni di BoW nel catturare il contesto. Le rappresentazioni più semplici sono gli interi unici o i vettori one-hot, ma questi non catturano le relazioni semantiche.

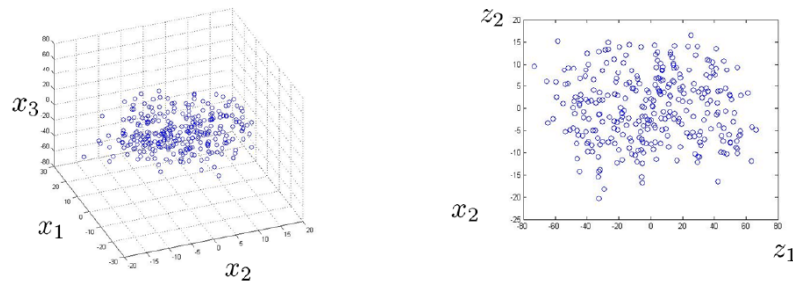
6 Riduzione della Dimensionalità

6.1 PCA

Il **Principal Component Analysis** è una tecnica per ridurre il numero di caratteristiche (dimensionalità) di un dataset, ed è utile per:

- **Compressione dei dati:** Riduce lo spazio di archiviazione e la sua complessità computazionale;
- **Visualizzazione dei dati:** Permette di visualizzare dati ad alta dimensione 2D o 3D;
- **Accelerazione dell'apprendimento supervisionato:** Rimuove caratteristiche ridondanti o meno informative;

Reduce data from 3D to 2D

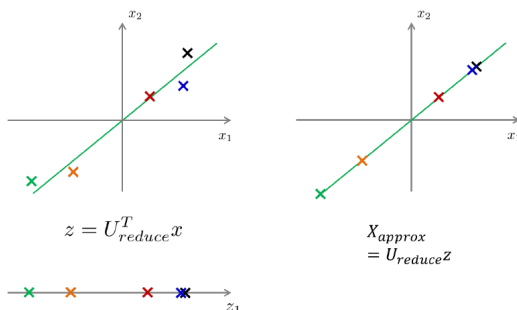


PCA non è un regressore lineare, anche se visivamente può sembrare simile, in quanto PCA cerca di minimizzare la distanza ortogonale dai dati alla componente principale riducendo gli errori di proiezione, mentre la regressione minimizza l'errore verticale dell'ipotesi; Per poter minimizzare la distanza e l'errore di proiezione PCA agisce in questo modo:

1. Presi i dati li si scala per normalizzarli con la *mean normalization*, *feature scaling* ...;
2. Si calcola la matrice di covarianza per confrontare tutti i punti tra di loro ed evidenziare eventuali somiglianze;
3. Sfrutta gli autovettori e autovalori interni alla matrice sopra costruita per identificare le dimensioni o i vettori più importanti per rappresentare i dati;
4. Con i k vettori sopra ottenuti costruisce la matrice $U_{reduce} = U(:, 1 : k)$;
5. Infine ogni valore della U_{reduce} viene moltiplicato per ogni valore x_i , così facendo si trova la proiezione dei dati sulla nuova base;
6. I vettori quindi diventano la nuova base e gli elementi non saranno più con dimensione N ma K ;

Se i dati poi vengono ripoiettati sul sistema originale si nota che l'errore nella semplificazione porta ad avere tutti i punti sulla retta e non più distanti. Questo quindi porta ad una stima di ricostruzione.

Come regola generale si cerca di limitare al minimo il numero di ridimensionamenti onde evitare perdite di dati.



Formule:

- **Matrice di Covarianza:** $\Sigma = \frac{1}{m} \sum_{i=1}^n (x_i)(x_i)^T$
- **Decomposizioni valori singoli:** $[U, S, V] = \text{svd}(\Sigma)$
- **Proiezione k autovettori:** $U_{reduce} = U(:, 1 : k)$
- **Proiezione su nuovo spazio z:** $z = U_{reduce}^T x$
- **Ricostruzione dato originale:** $x_{approx} = U_{reduce} z$

6.2 Scelta di k

Spesso viene calcolato l'**average squared projection error**, tipicamente un buon k porta ad avere una varianza sotto l'1%.

Formula:

- **Varianza errore di proiezione medio:** $\frac{\frac{1}{m} \sum_{i=1}^m \|x^i - x_{approx}^i\|^2}{\frac{1}{m} \sum_{i=1}^m \|x^i\|^2} \leq 0.01$
- **Varianza semplificata:** $1 - \frac{\sum_{i=1}^k S_{ii}}{\sum_{i=1}^N S_{ii}}$

7 Rilevamento delle Anomalie

Questa è una tecnica per **identificare** eventi *rari* o *anomalie* in un dataset. A differenza dell'apprendimento supervisionato, dove si hanno molti esempi di classi normali e anomale, nel rilevamento delle anomalie si hanno prevalentemente dati "normali" e pochissimi esempi di anomalie.

Si basa sulla **stima della densità** dei dati, spesso assumendo una **distribuzione Gaussiana** (normale) per le caratteristiche. Punti con una probabilità molto bassa di essere osservati sono considerati anomalie.

Si procede estraendo le features, per creare un modello ed identificare gli esempi che stanno dentro o fuori dalla probabilità determinata.

Formule:

- **Media** per feature(j): $\mu_j = \frac{1}{m} \sum_{i=1}^m x_j^i$
- **Varianza** per feature (j): $\sigma_j^2 = \frac{1}{m} \sum_{i=1}^m (x_j^i - \mu_j)^2$
- **Distribuzione gaussiana univariata:** $p(x) = \frac{1}{\sigma \sqrt{2\pi}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$
- **Multivariata:** $p(\vec{x}) = \prod_{j=1}^n p(x_j; \mu_j, \sigma_j^2)$
(assumendo indipendenza tra feature)

